AI 协议 · Agent · 工具调用 · 上下文工程

MCP：让模型从「会说」到「能做事」

一份给非协议工程师的 MCP 研究笔记

MCP 是一套开放协议，让 AI 应用用统一方式连接外部工具、数据和服务；它不生产智能，只负责把上下文送进模型，把动作送出去。

阅读	来源	可信度
18 分钟	7 个	较强

目录

01 / 先把定义说窄一点 . . . 3
02 / 它从哪来：一次「接口税」的集体止痛 . . . 3
为什么是现在，而不是更早？ . . . 3
03 / 它怎么运作：三层、三类原语、两种传输 . . . 4
三层：谁负责什么 . . . 4
三类原语：做事、读料、走流程 . . . 5
两种传输：本地与远程 . . . 5
04 / 它在版图里的位置：不是替代，而是叠加 . . . 5
和 function calling：一个管「说」，一个管「接」 . . . 6
和 RAG：连接协议不解决检索质量 . . . 6
05 / 证据与争议： adoption 很快，治理还在补课 . . . 6
最强的反方：它可能只是一个「可选的插件通道」 . . . 7
另一个争议：USB-C 类比掩盖了什么 . . . 7
争议场 . . . 8
06 / 对你意味着什么：三类人的下一步 . . . 8
如果你是开发者 . . . 8
如果你是普通用户 . . . 9
如果你只是想学明白 . . . 9
07 / 怎么继续学：一条七天的最小路径 . . . 9
延伸阅读 . . . 9
08 / 方法与局限 . . . 10
可信度说明 . . . 11
来源清单 . . . 12

本篇读物由问底流程生成。带编号的上标指向来源清单；争议场保留最强反方，而不是稻草人。

一句话定义

01 / 先把定义说窄一点

MCP（Model Context Protocol）是一套开放协议，规定 AI 应用如何发现、调用外部工具，如何读取外部资源，如何复用预置的工作流。它不训练模型，也不替模型思考；它解决的是「模型如何可靠地接触到模型之外的东西」。[2]

> 关键洞察
>
> 一句话记住
>
> Function calling 让模型「会点菜」；MCP 让整个应用「有一套统一的点菜、上菜和后厨接口」。一个管表达，一个管连接。

这个区分很重要。很多关于 MCP 的讨论把它和「模型能力」混在一起，仿佛接上 MCP，模型就突然会查数据库、会发邮件、会操作浏览器。实际上，模型仍然只会生成文本或结构化的调用意图；真正执行动作的是宿主应用和它背后的工具。MCP 做的是把「有哪些工具、怎么调用、返回什么」这件事标准化。所以，MCP 的第一层价值是降低集成成本，第二层价值才是让能力可组合。如果只看到第一层，它像一个更规范的插件系统；看到第二层，它才像 AI 应用的基础设施。

纵向脉络

02 / 它从哪来：一次「接口税」的集体止痛

在 MCP 出现之前，AI 应用接工具的方式基本是「一家一议」：每个模型提供方有自己的 function calling 格式，每个应用有自己的工具注册方式，每个数据源又有自己的鉴权和返回结构。结果是一个典型的 N×M 问题——N 个模型、M 个工具，就要维护 N×M 套适配。[1]

Anthropic 在 2024 年 11 月发布 MCP 时，用了一个很聪明的类比：把它想成 AI 应用的 USB-C 接口。这个类比不一定严谨，但它准确抓住了痛点——大家不想再为每一种组合重新焊一根线。[1]

> “把 MCP 想成 AI 应用的 USB-C 接口：不是让设备更聪明，而是让设备之间更容易接上。”
>
> —— 对 Anthropic 发布公告中类比的一种转述

但 MCP 并不是第一个尝试。更早的插件体系、各家 agent 框架的工具抽象、以及模型侧的 function calling，都在解决相邻的问题。MCP 的时机优势在于三件事同时发生：agent 从演示走向真实工作流；工具调用成为模型的基础能力；以及一个开放、可自托管、跨语言的协议正好填上了「应用与工具之间」的空缺。

为什么是现在，而不是更早？

- 需求侧：AI 应用开始需要读文件、查数据库、调 API、操作浏览器，单靠 prompt 已经不够。

- 供给侧：模型侧的 function calling 逐渐稳定，应用侧需要一层比「每家自己写适配」更通用的连接方式。
- 生态侧：开源实现和自托管 server 降低了参与门槛，协议不必先等某个巨头把平台做出来。

这也解释了为什么它先由 Anthropic 推动，后来 OpenAI 的 Agents SDK、以及大量 IDE 与桌面应用开始支持：大家不一定想在协议层站队，但都希望少写一点重复的适配代码。[45]

机制拆解

03 / 它怎么运作：三层、三类原语、两种传输

MCP 的结构可以先用一句话概括：宿主（host）里运行客户端（client），客户端连接服务器（server），服务器把外部系统的能力暴露成三类原语。[2]

节点	关系	说明
MCP	—	概念中心
Host / 宿主	管理会话、模型与权限	与中心的连接
Client / 客户端	与 server 一对一连接	与中心的连接
Server / 服务器	包装外部能力	与中心的连接
Tools / 工具	可执行的动作	与中心的连接
Resources / 资源	可读取的上下文	与中心的连接
Prompts / 提示词	预置工作流	与中心的连接
Transport / 传输	stdio 或流式 HTTP	与中心的连接
外部系统	文件、数据库、API	与中心的连接

表 1　MCP 的结构。宿主管理体验与权限，客户端负责一对一会话，服务器把外部能力包装成三类原语。

三层：谁负责什么

角色	它是什么	它负责什么
Host	你直接使用的 AI 应用，例如桌面助手、IDE、agent 工作台	管理会话、模型、权限和用户体验；决定要不要连接某个 server
Client	Host 内部与某个 server 保持一对一会话的连接器	协商能力、转发请求、处理通知与生命周期
Server	独立进程或远程服务，包装文件、数据库、API 等外部能力	声明自己提供什么工具/资源/提示词，并执行调用

把 server 想成「带权限的适配器」比想成「插件」更准确。插件通常运行在宿主内部，权限边界模糊；MCP server 可以是一个独立进程、一个远程服务，甚至是别人写的第三方代码。它带来的能力越大，越需要被当作第三方依赖来对待。

三类原语：做事、读料、走流程

- Tools（工具）：模型可以发起的动作，例如查询数据库、发送请求、创建文件。工具通常有输入 schema，调用结果会回到模型上下文。
- Resources（资源）：可以被读取的上下文，例如文件、记录、网页内容。资源更像「资料」，不一定产生副作用。
- Prompts（提示词）：由 server 预置的工作流或模板，让用户/宿主以一致方式发起某类任务。

> 补充说明
>
> 一个容易被忽略的细节
>
> MCP 不只让模型「调用工具」。它还允许 server 在运行过程中发送通知、请求用户确认，甚至发起采样请求。能力越双向，信任边界越重要。

两种传输：本地与远程

本地 server 常用 stdio 传输：宿主启动一个子进程，通过标准输入输出交换 JSON-RPC 消息。远程 server 则使用可流式 HTTP 传输，适合多用户、集中部署和统一鉴权。[3]两种方式共享同一套消息语义，但部署与安全模型完全不同：本地 server 更像「你电脑上的一个程序」，远程 server 更像「一个你需要登录和审计的第三方服务」。

协议本身用 JSON-RPC 组织请求、响应和通知，连接开始时双方会协商能力与版本。这个设计不复杂，真正的复杂度在「能力协商之后」：谁有权调用、调用结果是否可信、出错时谁负责。

横向对比

04 / 它在版图里的位置：不是替代，而是叠加

把 MCP 和 function calling、插件、RAG 放在一张表里，最容易产生的误解是「谁取代谁」。它们其实处在不同层：function calling 是模型表达调用意图的方式；插件是某个平台把外部服务接进来的机制；RAG 是一种把资料检索进上下文的模式；MCP 则是应用与工具之间的连接协议。

	它解决什么	谁来做适配	典型风险
MCP	跨应用、跨进程连接工具与数据	server 作者 + 宿主客户端	
Function calling	让模型发出结构化调用意图	模型提供方 + 应用	
插件	把外部服务接进某个平台	平台 + 服务方	
RAG	把相关资料检索进上下文	应用/检索管线	

表 2　四种「让模型连接外部」的思路。它们不在同一层：MCP 是连接协议，function calling 是模型能力，插件是平台机制，RAG 是检索模式。

■ 和 function calling：一个管「说」，一个管「接」

模型通过 function calling 生成一个结构化的调用请求，但「这个请求发给谁、用什么鉴权、返回结果怎么格式化」并不是 function calling 规定的事。MCP 把这些应用层的问题接了过去。所以它们不是竞争关系：MCP 客户端通常仍然依赖模型的 function calling 能力来决定何时调用哪个工具。

■ 和 RAG：连接协议不解决检索质量

RAG 的核心是「找对资料、放对上下文」；MCP 的核心是「用统一方式把资料和工具接进来」。一个 MCP server 可以暴露检索工具，但检索的召回率、切分策略、引用准确性仍然要单独设计。把 MCP 当成 RAG 的替代品，会让系统看起来更整齐，却不会让答案更可靠。

> 值得警惕
>
> 别把「能连上」当成「能信任」
>
> MCP 让连接变便宜，也让「接错东西」变便宜。一个来源不明的 server、一个权限过大的 token、一段被注入的工具描述，都可能让模型做出你没授权的动作。

反方与证据

05 / 证据与争议： adoption 很快，治理还在补课

从公开资料看，MCP 的采用速度确实快：Anthropic 发布后，MCP 客户端目录里出现了桌面应用、IDE、agent 框架等多类宿主；OpenAI 的 Agents SDK 也提供了 MCP 接入能力。[45]这说明它至少跨过了「只有一家在用」的门槛。



图 1　MCP 的关键节点。从发布到生态扩张：协议本身变化不算快，变化快的是宿主与 server 的数量。

但「采用快」不等于「成熟」。MCP 规范自己就把若干安全问题列为部署时必须处理的风险，包括混淆代理（confused deputy）、token 透传、会话劫持等；这些问题的共同点是：协议可以把消息传对，但不能替你做授权判断。[6]

■ 最强的反方：它可能只是一个「可选的插件通道」

反方最有力的论点不是「MCP 没用」，而是「MCP 可能停在开发者工具层」。如果主流客户端只把它当作众多连接方式之一，如果企业数据平台仍然要求私有连接器，如果每个大厂都在支持 MCP 的同时保留自己的私有能力，那么 MCP 会成为一种重要的兼容层，却未必成为统一入口。

这个反方不能被一句「生态会赢」打发。它提示我们区分两种成功：协议被采用，和协议成为默认。前者已经发生，后者还没有。

■ 另一个争议：USB-C 类比掩盖了什么

USB-C 类比之所以好用，是因为它把「接口统一」的收益讲得非常直观；但它也掩盖了 MCP 特有的问题：USB-C 不会在你插上线之后，自己决定读取哪个文件、向哪个地址发请求。MCP 连接的是有语义、有权限、有副作用的工具。接口统一只解决了「怎么接」，没有解决「接上之后能做什么」。

> 继续追问
>
> 判断一条 MCP 新闻值不值得关注
>
> 问三个问题：它增加了哪类宿主？它解决了什么权限/审计问题？它让哪个原本需要定制适配的场景变成可复用？如果三个都答不上来，可能只是「又接了一个工具」。

把最强反方放在台面上

争议场

争议 01

MCP 会成为 AI 应用的「USB-C」吗?

支持方

只要工具生态继续碎片化，一个开放、可组合、被多家客户端采用的连接协议就有网络效应。

- 它把 N×M 的定制适配变成 N+M 的公共接口，集成成本随生态扩大而下降。[1]
- 从 IDE、桌面 AI 应用到 agent 框架，客户端侧已经出现跨厂商采用，说明它不只是 Anthropic 的内部约定。[4][5]

反方

统一接口不等于统一生态；大厂完全可以在支持 MCP 的同时保留自己的私有能力与分发入口。

- 协议只规定「怎么连」，不规定「连上之后谁负责」；第三方 server 的权限、质量和安全无法由协议自动保证。[6]
- 工具描述、资源内容和模型输出都可能成为注入通道；连接越多，攻击面越大。[6]

目前的判断 · 置信度 中

更稳妥的判断是：MCP 有机会成为「工具连接层」的事实标准之一，但不会自动成为所有 AI 上下文的统一入口。它的上限取决于客户端覆盖面与安全治理，而不是协议本身有多优雅。

什么会改变这个判断：如果未来 12—18 个月内，主流的操作系统、浏览器和企业数据平台都把 MCP 作为一等公民，并且出现可审计的权限与供应链标准，我会把判断上调。反之，如果多数客户端只把它当「可选的插件通道」，它就会停留在开发者工具层。

落地建议

06 / 对你意味着什么：三类人的下一步

如果你是开发者

先把 MCP 当成「带权限的适配层」，不要当成「万能插件」。第一个 server 建议从只读能力开始：读一个目录、查一张表、取一个网页的正文。只读 server 能让你把协议、传输、错误处理和权限边界跑通，而不会一上来就把危险动作交给模型。

01 跑通一个最小 server

用官方 SDK 暴露一个只读工具，确认 host 能发现、调用并拿到结构化结果。

02 加上边界

限制可访问的路径/表/域名，给每次调用打日志，明确哪些动作需要用户确认。

03 再谈组合

当你有一个稳定、可审计的 server 之后，再考虑把多个 server 组合成工作流。

如果你是普通用户

你不需要理解 JSON-RPC，但需要建立一个直觉：连接一个 MCP server，等于把一个外部程序接进你的 AI 助手。只连接你信任来源的 server；对「读取全部文件」「访问所有网页」「代你发消息」这类权限保持警惕；重要操作要求二次确认。

如果你只是想学明白

不要从「写一个万能连接器」开始。先读官方文档里的 host/client/server 模型，再跑一个示例 server，观察一次完整的请求与响应；最后读安全最佳实践，理解为什么「协议正确」不等于「系统安全」。这条路径比背概念更能建立判断力。

先写一个只读 server，再谈万能连接器。

学习路径

07 / 怎么继续学：一条七天的最小路径

下面这条路径假设你会一点编程，但没接触过 MCP。目标不是成为协议专家，而是能独立判断一个 MCP 方案是否靠谱。

01 第 1 天：读模型

只读官方文档的 host/client/server 与三类原语，画出你自己的结构图。

02 第 2 天：跑示例

跑通一个官方示例 server，用宿主连接它，观察工具列表与调用结果。

03 第 3 天：看消息

抓一次 JSON-RPC 请求与响应，理解 initialize、tools/list、tools/call 的顺序。

04 第 4 天：加权限

给示例 server 加上路径或域名白名单，故意触发一次拒绝，观察错误如何回到模型。

05 第 5 天：读安全

读规范的安全最佳实践，列出你认为最重要的三个风险。

06 第 6 天：做一个小工具

写一个只读 server，解决你自己的一个小问题，例如查本地笔记或读一个公开 API。

07 第 7 天：写复盘

用 500 字回答：MCP 帮你省掉了哪段适配？又引入了哪类新风险？

延伸阅读

- Anthropic 的 MCP 发布公告：理解它最初想解决什么问题。[1]
- MCP 官方文档的架构与概念页：建立 host/client/server 的准确模型。[2]
- MCP 规范与安全最佳实践：理解协议边界与部署风险。[3][6]
- OpenAI Agents SDK 的 MCP 文档：看另一个生态如何接入。[4]
- MCP 客户端目录：观察真实宿主如何分布。[5]

研究说明

08 / 方法与局限

这份样例使用了纵轴—横轴、第一性原理和钢人论证三副透镜：纵轴看它如何从「接口碎片化」中长出来，横轴把它和 function calling、插件、RAG 放在同一张桌上，钢人论证则把支持和反对 MCP 的最强理由都摆出来。

主要局限有三个。第一，公开资料以官方文档、规范和项目仓库为主，独立实证研究仍然偏少；第二，采纳规模缺少统一口径的量化数据，本报告只能描述方向，不能给出精确份额；第三，安全风险在快速演变，报告中的风险清单不是穷举。

> 补充说明
>
> 证据强度
>
> 关于「MCP 是什么、怎么运作」的部分证据较强，因为有一手规范与官方文档；关于「它会不会成为统一标准」的部分属于判断，置信度中等，需要持续观察。

Credibility · 5 signals

可信度说明

整体可信度较高；五个信号里最需要留意的是「反方强度」。

信号	状态	怎么看
来源硬度	较强	7 个来源、2 种类型；6 个高可信，7 个已打开正文。事实性段落均带引用；判断性段落明确标注为分析。
反方强度	中等	1 场正反辩论，0 个争议条目。安全与治理的批评主要来自规范自身，独立第三方研究偏少。
推断透明度	较强	证据账本未随读物保存；事实性段落均带引用；判断性段落明确标注为分析。
未知与争议	较强	记录了 3 个证据缺口、3 条局限、0 个争议。
时效与可证伪	较强	7 个来源有明确日期，7 个在近三年内；1 个争议写明了什么会改变判断。

Sources · 7

来源清单

01 Introducing the Model Context Protocol

官方/规范 · 官方 / 法规 · Anthropic · 2024-11-25

打开原文 · anthropic.com

02 MCP Documentation — Introduction

官方/规范 · 官方 / 法规 · Model Context Protocol · 2026-01-01

打开原文 · modelcontextprotocol.io

03 MCP Specification (2025-06-18)

官方/规范 · 官方 / 法规 · Model Context Protocol · 2025-06-18

打开原文 · modelcontextprotocol.io

04 OpenAI Agents SDK — MCP

官方/规范 · 官方 / 法规 · OpenAI · 2025-03-11

打开原文 · openai.github.io

05 MCP Clients

官方/规范 · 官方 / 法规 · Model Context Protocol · 2026-01-01

打开原文 · modelcontextprotocol.io

06 MCP Security Best Practices

官方/规范 · 官方 / 法规 · Model Context Protocol · 2025-06-18

打开原文 · modelcontextprotocol.io

07 modelcontextprotocol/modelcontextprotocol

代码仓库 · 学术 / 研究 · GitHub · 2024-11-25

打开原文 · github.com